integrated into the application or a system like an API gateway, reverse proxy, or backend service. Its role is to intercept requests and call the policy engine to determine if access should be granted. From the caller's perspective, authorization happens in real time and is closely tied to the specific action being performed.

Table 6.4 summarizes how the four core components—PEP, PDP, PAP, and PIP—are usually realized in ABAC systems.

Table 6.4 Mapping ABAC onto the authorization reference architecture

Component	What It Does	How ABAC Implements It
Policy Enforcement Point (PEP)	Intercepts access requests and enforces the decision	Typically integrated into the application, API gateway, or middleware layer that intercepts requests and queries the PDP
Policy Decision Point (PDP)	Determines whether the requested action is allowed	A policy engine (like Cedar, Open Policy Agent, or a homegrown system) that evaluates code-like policies against the current attributes
Policy Administration Point (PAP)	Manages and stores roles, permissions, and assignments	Maintained as code, often versioned alongside application code or kept in centralized policy stores. IDEs or CI/CD tools may be used to manage changes
Policy Information Point (PIP)	Supplies user and role data to the PDP at runtime	Pulls attributes from identity providers, HR systems, resource metadata, device management platforms, and contextual signal providers (e.g., time, risk, location)

The Policy Decision Point (PDP) is usually implemented as a policy engine that evaluates policies. Unlike RBAC, where decisions rely on simple role-to-permission mappings, ABAC policies are more expressive: they're code-like logic that references input attributes. The PDP receives the request, fetches the relevant attributes, retrieves the relevant policies, and evaluates them to return an allow or deny decision.

In practice, ABAC PDPs must balance flexibility with performance. Because policies can number in the hundreds or thousands, efficient retrieval and